

2026년 전기 졸업과제 중간보고서

KD-CNN 기반 경량 서비스 메시지를 활용한
클라우드 네이티브 침입탐지시스템 설계 및 구현



부산대학교 정보컴퓨터공학부

지도교수: 최윤호

팀 명: DeepMesh

(suc2150@pusan.ac.kr) 신의철 202014325

(ppvws@pusan.ac.kr) 정의진 202055598

(siiihaaa@pusan.ac.kr) 이시하 202322585

목차

1. 과제 목표	3
1-1. 과제 배경	3
1-2. 과제 세부 목표	3
2. 요구사항 및 제약사항 반영에 따른 구현 수정 사항	4
2-1. 기존 요구사항 명세에 대한 수정사항	4
2-2. 제약사항 분석 및 대안	6
3. 보고 시점까지의 수행 내용 및 중간 결과	7
3-1. MSA 실험환경 개발	7
3-2. 쿠버네티스 클러스터 구성	15
3-3. 쿠버네티스 배포	16
3-4. 트래픽 데이터 수집	17
3-5. 모델 평가	19
4. 남은 개발 단계 설계 상세화	22
4-1. remote attack과 escape 시나리오의 추가로 공격 기법 커버리지 확장	22
4-2. 반복 실행으로 통계적 변동성 보강	22
4-3. 프록시 사이드카 및 이상 탐지 모듈 탑재	23
4-4. 트래픽 후속 처리 및 control plane 교차 검증	24
4-4. 실시간 모니터링 대시보드	25
5. 전체 시스템 통합 구조	26
6. 구성원별 개발 진척도 및 향후 추진 계획	27
6-1. 중간보고서 작성 시점 개발 진척도	27
6-2. 중간보고서 이후 향후 추진 계획	27
7. 참고 문헌	29

1. 과제 목표

1-1. 과제 배경

컨테이너 가상화 기술이 발전하면서 클라우드 네이티브 방식은 현대 애플리케이션을 구축하고 배포하는 지배적인 방법론으로 자리잡았다. 특히, Kubernetes를 기반으로 여러 컨테이너가 상호 연결되어 동작하는 구조를 통해 애플리케이션을 상황에 따라 유연하게 확장하고 운영할 수 있다.

그러나 클라우드 네이티브 환경의 규모가 커지고 복잡해짐에 따라 보안 위협 또한 고도화되고 있다. 특히, 최근에는 마이크로서비스 아키텍처에서 하나의 컨테이너에 발판을 마련한 공격자가 내부 서비스 간 통신(east-west)을 통해 인접 서비스로 피해를 확산시키는 수평 이동의 공격 방법이 등장하고 있다. 이러한 공격은 정상 사용자 요청과 목적지 및 엔드포인트가 겹치는 경우가 많아 개별 요청 단위로는 식별이 어렵고, 각 서비스가 평소 주고받는 고유한 트래픽 패턴에서 얼마나 벗어나는지를 관찰해야만 탐지할 수 있어 기존의 시스템으로는 공격을 탐지하기 어렵다.

본 과제에서는 이러한 기존의 정적 규칙 분석 방식의 한계를 해결하기 위해 CNN을 이용해 공격의 컨텍스트를 파악하고, Knowledge Distillation을 이용해 이상탐지 모듈을 경량화 한 후 이를 Kubernetes 클러스터의 각 Pod에 사이드카 프록시 형태로 탑재하여 정적 규칙에 의존하지 않고 서비스별 트래픽 패턴을 학습해 CPU 환경에서 실시간으로 동작하는 클라우드 네이티브 침입탐지 시스템을 구현하는 것을 목표로 한다.

1-2. 과제 세부 목표

앞선 배경과 문제 분석을 바탕으로 본 과제에서는 다음 다섯 가지를 세부 목표로 설정한다.

① Kubernetes Cluster에 MSA 실험 환경 구축

실제 서비스 트래픽과 공격 트래픽이 발생하는 환경을 제공한다.

② KD-CNN + OCSVM 기반 이상 탐지 모듈 구현

Knowledge Distillation 방법을 이용해 CPU로 동작하는 경량 모델을 구현한다. 이를 One-Class SVM과 결합하여 레이블 없는 정상 트래픽만으로 비지도 이상 탐지를 수행한다.

③ 사이드카 기반 경량 서비스 메시 구현

프록시 컨테이너를 사이드카로 결합하여, 트래픽을 인터셉트하고 이상탐지를 수행한다.

④ 트래픽 후속 처리 기능 구현

이상 탐지에 따라 악성 요청을 차단하거나, 정상 ReplicaSet Pod의 응답으로 대체한다.

⑤ 실시간 모니터링 대시보드 개발

시스템 현황을 topology 그래프 형태로 시각화하는 관리자용 대시보드를 구축한다.

2. 요구사항 및 제약사항 반영에 따른 구현 수정 사항

2-1. 기존 요구사항 명세에 대한 수정사항

아래 표는 시스템이 제공해야 할 기능적 요구사항과 비기능적 요구사항을 정리한 표이다. 굵게 표시된 부분은 중간보고서 작성 시점까지 개발을 진행하면서 추가 및 수정된 요구사항을 의미한다.

• 기능적 요구사항

대분류	소분류	코드	요구사항
사용자 인증 및 인가	회원가입	FR-01	사용자는 아이디와 비밀번호를 사용해 계정을 등록할 수 있어야 한다.
	로그인	FR-02	사용자는 등록된 자격증명으로 시스템에 로그인하여 Access Token을 응답으로 발급받고, Refresh Token을 쿠키로 발급받을 수 있어야 한다.
	로그아웃	FR-03	인증된 사용자는 시스템에서 로그아웃해서 인증 세션을 종료할 수 있어야 한다.
	인가	FR-04	인증된 사용자만 게시글 작성, 수정, 삭제, 댓글 작성, 수정, 삭제에 접근할 수 있어야 한다.
게시글 관리	목록 조회	FR-05	사용자는 인증 없이 게시글 목록을 조회할 수 있어야 한다.
	상세 조회	FR-06	사용자는 인증 없이 특정 게시글의 상세 내용과 관련 댓글 목록을 함께 조회할 수 있어야 한다.
	작성	FR-07	인증된 사용자는 새 게시글을 작성할 수 있어야 한다.
	수정	FR-08	인증된 사용자는 자신이 작성한 게시글을 수정할 수 있어야 한다.
	삭제	FR-09	인증된 사용자는 자신이 작성한 게시글을 삭제할 수 있어야 하며, 삭제 시 해당 게시글에 속한 댓글도 함께 삭제되어야 한다.
댓글 관리	목록 조회	FR-10	사용자는 인증 없이 특정 게시글의 댓글 목록을 조회할 수 있어야 한다.
	작성	FR-11	인증된 사용자는 게시글에 댓글을 작성할 수 있어야 한다.
	수정	FR-12	인증된 사용자는 자신이 작성한 댓글을 수정할 수 있어야 한다.
	삭제	FR-13	인증된 사용자는 자신이 작성한 댓글을 삭제할 수 있어야 한다.
이상 탐지	실시간 감시	FR-14	시스템은 서비스 간 트래픽을 세션(5-tuple) 단위로 수집하고 5-패킷 슬라이딩 윈도우로 분할하여, 메인 컨테이너의 요청 및 응답을 실시간으로 감시, 탐지할 수 있어야 한다.
	비지도 학습	FR-15	시스템은 공격 트래픽 레이블 없이 정상 트래픽만으로 이상 탐지 모델을 구축해야 한다.
	서비스별 탐지	FR-16	시스템은 서비스별로 독립적인 정상 행동 기준을 학습하여 서비스 특화 이상 탐지를 수행할 수 있어야 한다.

	요청 차단	FR-17	시스템은 이상으로 판별된 요청 트래픽 중 Request Verifier가 다른 Pod에서 관측된 적 없는 요청으로 판정한 경우 이를 차단할 수 있어야 한다.
	응답 대체	FR-18	시스템은 이상으로 판별된 응답 트래픽에 대해 동일 ReplicaSet의 다른 Pod 응답과 내용을 비교하여, 내용이 다를 경우 신뢰할 수 있는 정상 응답으로 대체할 수 있어야 한다.
	정상 전달	FR-19	시스템은 정상으로 판별된 트래픽을 목적지로 지연 없이 전달할 수 있어야 한다.
	교차 검증	FR-20	시스템은 이상 탐지 1차 판별 결과에 대해 ReplicaSet 간 교차검증을 수행하여 False Positive를 감소시켜야 한다.
모니터링	topology 시각화	FR-21	관리자는 서비스 간 트래픽 흐름과 각 서비스의 이상 탐지 상태를 topology 그래프 형태로 실시간으로 확인할 수 있어야 한다.
	탐지 현황	FR-22	관리자는 탐지 건수, 처리 결과, 서비스별 상태를 요약한 현황 정보를 실시간으로 확인할 수 있어야 한다.
	이력 조회	FR-23	관리자는 탐지 시각, 대상, 서비스, 처리 결과를 포함한 이상 탐지 이력을 시간 순으로 조회할 수 있어야 한다.
	실시간 알림	FR-24	시스템은 이상 탐지 이벤트 발생 시 관리자 화면에 실시간으로 알림을 전달할 수 있어야 한다.
데이터 수집	정상 트래픽 수집	FR-25	시스템은 정상적인 사용자 행동 시나리오를 자동으로 재현하여 이상 탐지 모델 학습용 정상 트래픽 데이터를 수집할 수 있어야 한다.
	공격 트래픽 재현	FR-26	시스템은 MITRE ATT&CK 프레임워크 기반의 다양한 공격 시나리오를 재현하여 이상 탐지 성능을 검증할 수 있어야 한다.

• 비기능적 요구사항

분류	번호	요구사항
성능	NF-01	이상 탐지가 활성화된 상태에서 end-to-end 응답 지연은 50ms 이하여야 한다.
	NF-02	이상 탐지가 활성화된 상태에서 시스템은 300 req/s 이상의 처리량을 유지한다.
자원 제약	NF-03	이상 탐지 모델은 GPU 없이 단일 vCPU 환경에서 실시간 추론이 가능해야 한다.
	NF-04	탐지 프록시는 메인 애플리케이션 코드의 변경 없이 사이드카 패턴으로 결합되어야 한다.
탐지 정확성	NF-05	이상 탐지 모델의 F1-score는 80% 이상을 달성한다.
	NF-06	이상 탐지 모델의 ROC-AUC는 85% 이상을 달성한다.
가용성	NF-07	False-Positive 발생 시에도 Relay 매커니즘을 통해 서비스 중단 없이 정상 응답을 유지한다.
보안	NF-08	시스템은 침해된 컨테이너로부터 발생하는 서비스 간 Lateral Movement 공격을 탐지하고 차단할 수 있어야 한다.
	NF-09	각 서비스는 자신에게 할당된 데이터 저장소 외에 타 서비스의 데이터 저장소에 접근할 수 없어야 한다.

2-2. 제약사항 분석 및 대안

① MSA 실험 환경의 한계

- 본 과제 방식에서는 모델이 포함된 프록시 컨테이너를 각 서비스 Pod에서 개별 운영하고, Replicas Pod가 항상 필요하므로 자원 사용이 서비스 수에 비례해 증가해 중앙 집중형이나 정적 규칙 분석 방식에 비해 오버헤드가 크다. 따라서 내부 통신 보안 요구가 높은 도메인인 금융, 의료, 발전 등에서 MSA로 시스템을 설계할 때 우선 가치가 크다고 볼 수 있다.
- 그러나 현재 과제에서 가용한 Kubernetes 클러스터의 자원제약의 한계와 실질적인 기한 내 구현 가능성을 고려했을 때 실험환경을 단순화할 필요성이 존재했다.
- 이러한 점을 고려하여 본 과제에서는 가장 일반적인 CRUD 기능을 포함하는 게시판을 MSA로 설계하여 실험환경을 구축했다.

② 단일 공격 트래픽 탐지 한계

- 본 과제의 탐지 모델은 5개 패킷을 슬라이딩 윈도우로 묶어 5×1479 이미지로 변환한 뒤 이상 여부를 판별한다. 이 방식은 순차 열거나 반복 인증처럼 여러 패킷에 걸쳐 시간적 패턴이 드러나는 공격에는 효과적이지만, 단일 요청 하나로 완결되는 공격에는 취약하다. 예를 들어 SQL 삽입이나 위조된 토큰을 실은 단발성 요청은 5-패킷 시퀀스 안에서 뚜렷한 이상 패턴을 형성하지 못해 이미지 기반 탐지만으로는 놓칠 수 있다.

③ 트래픽 데이터 수집의 한계

- 각 Pod의 마이크로서비스 별 탐지 모델이 적용되어야 하므로 서비스 별 고유한 트래픽이 학습 데이터로 사용되어야 한다. 따라서 공개된 쿠버네티스 트래픽 데이터를 학습 용도로 사용할 수 없고, 실험 환경에서 직접 트래픽을 발생시켜 tcpdump를 이용해 캡처하는 방식으로 수집해야 한다.
- 또한, 수집된 데이터가 실험 환경에서 생성한 트래픽에 국한된다는 근본적 한계가 있다. 본 과제의 정상 트래픽은 회원가입, 로그인, 게시글 및 댓글 CRUD 시나리오를 자동 반복 실행하여 수집하므로, 실제 운영 환경의 다양한 사용자 행동 분포를 완전히 반영하지는 못한다.

④ 공격 시나리오 다변화 한계

- 실제 클라우드 네이티브 환경에서 발생 가능한 공격은 매우 다양하나, 본 과제는 실험 환경의 재현성과 기한 내 구현 가능성을 고려하여 대표적인 공격 유형으로 시나리오를 한정하였다. 이는 프록시 사이드카가 실제로 관측하는 트래픽이 서비스 간 east-west 통신이라는 점을 반영하여, 프록시가 탐지 가능한 대상으로 시나리오를 좁힌 것이다.

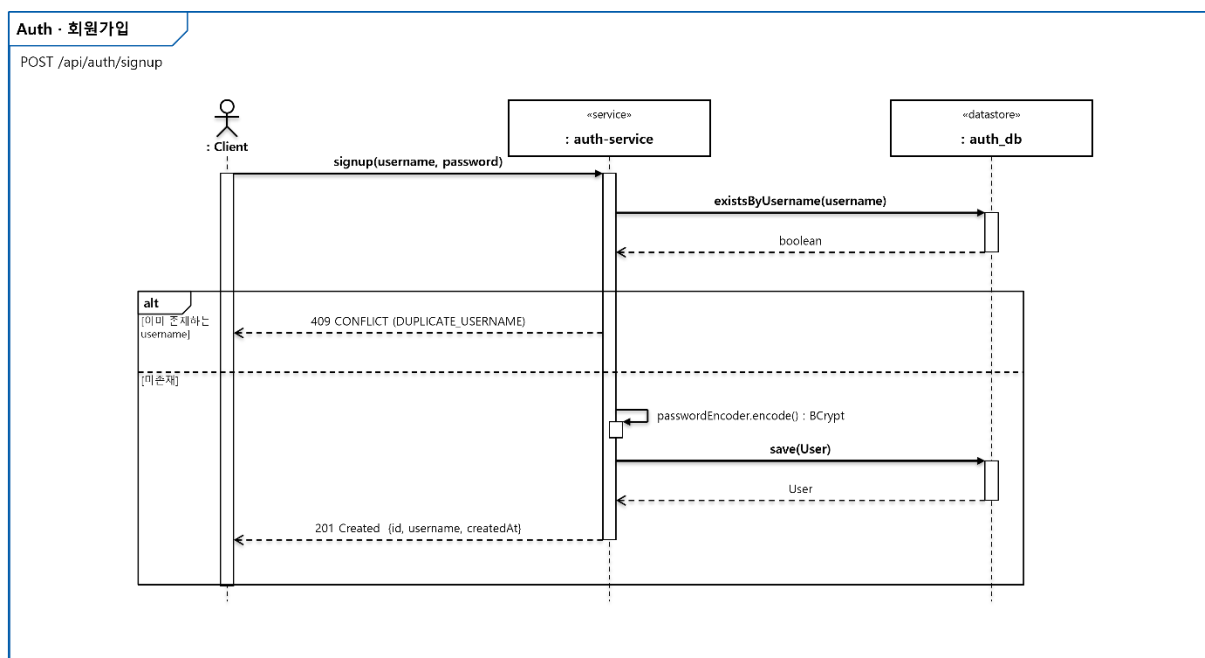
3. 보고 시점까지의 수행 내용 및 중간 결과

3-1. MSA 실험환경 개발

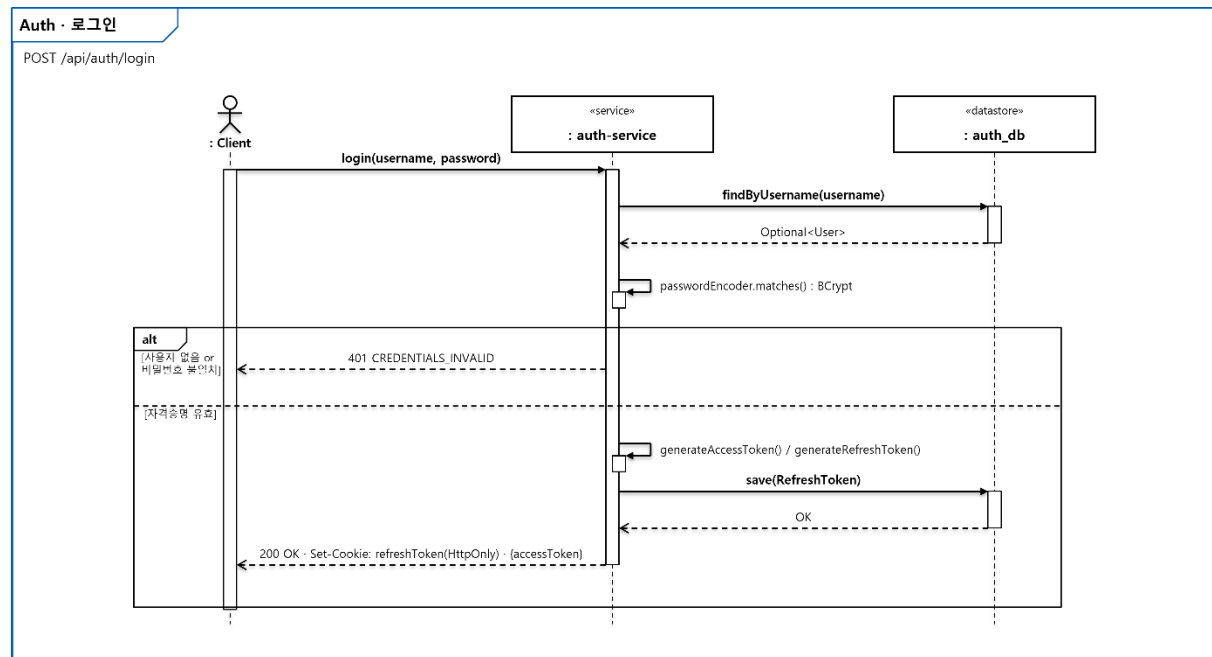
침입 탐지 모델의 학습·검증에 필요한 정상, 공격 트래픽을 발생시키기 위해, 가장 일반적인 CRUD 기능을 갖춘 게시판 애플리케이션을 마이크로서비스 아키텍처(MSA)로 구현하였다. Auth Service, Post Service, Comment Service 세 서비스로 도메인을 분리하고 각 서비스가 독립된 데이터베이스(auth_db, posts_db, comments_db)를 소유하도록 설계했다.

각 서비스는 Spring Boot 3.5, Java 17로 구현하였으며, 서비스 간 통신은 외부에 노출하지 않는 /internal/** 엔드포인트를 통한 동기식 REST 호출로 이루어진다. 특히 인가는 프레임워크가 아니라 Auth Service의 토큰 검증(/internal/auth/validate)에 위임하는 방식으로 처리하여, 서비스 간 (east-west) 내부 통신이 실제로 빈번히 발생하도록 하였다. 아래에서는 각 기능의 동작을 시퀀스 다이어그램으로 그 흐름을 설명한 것이다.

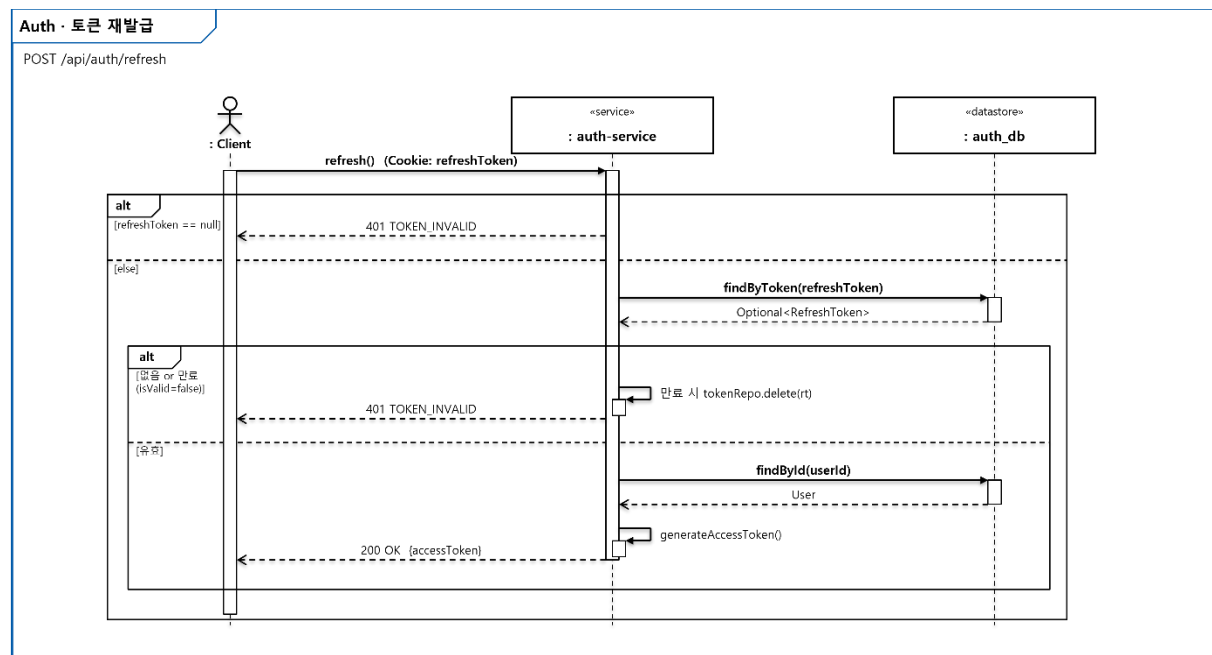
① 사용자 인증 및 인가 (Auth Service)



사용자가 아이디, 비밀번호로 회원가입을 요청하면 auth-service는 먼저 해당 아이디의 중복 여부를 조회한다. 이미 존재하는 경우 409 CONFLICT를 반환하고, 그렇지 않으면 비밀번호를 BCrypt로 해시하여 사용자 정보를 저장하고 생성된 계정 정보를 201 Created로 응답한다.

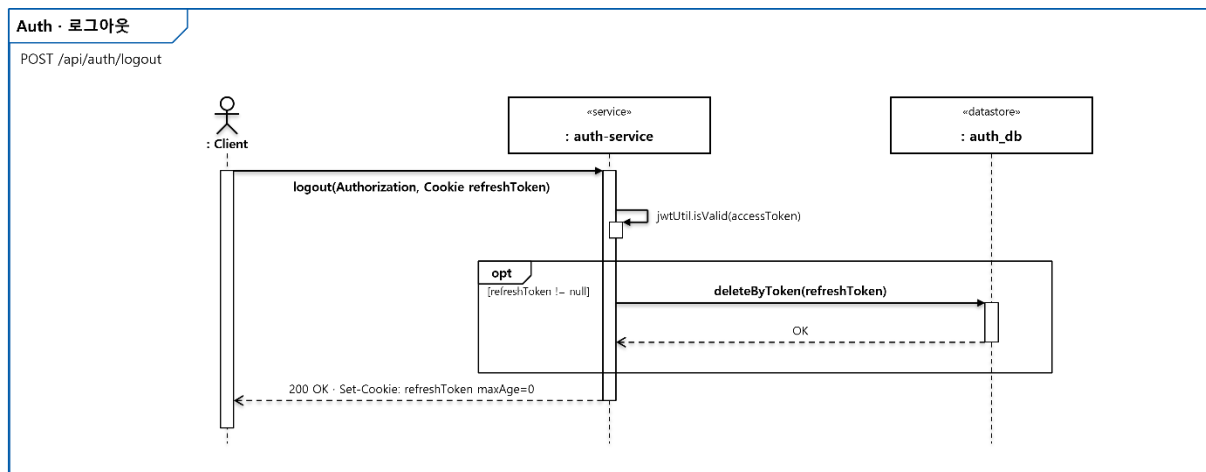


로그인 요청 시 아이디로 사용자를 조회한 후 BCrypt로 비밀번호 일치 여부를 검증한다. 사용자가 없거나 비밀번호가 일치하지 않으면 401(CREDENTIALS_INVALID)을 반환한다. 자격증명이 유효하면 Access Token과 Refresh Token을 발급하고, Refresh Token을 DB에 만료 시각과 함께 저장한다. 이후 Access Token은 응답 본문으로, Refresh Token은 HttpOnly 쿠키로 전달한다.

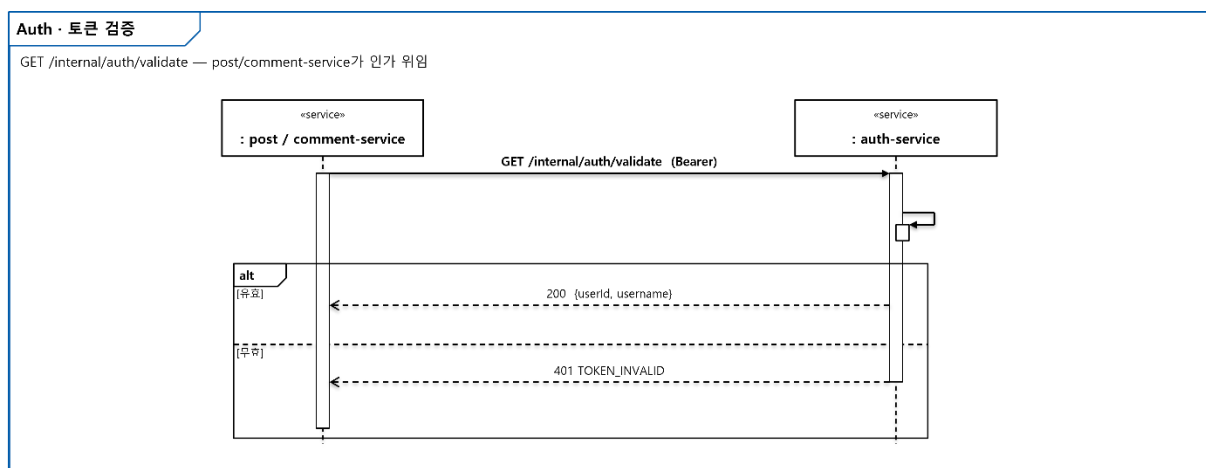


Access Token 만료 시 클라이언트는 쿠키에 담긴 Refresh Token으로 재발급을 요청한다. auth-service는 갱신 토큰의 존재 여부, DB 저장 여부, 서명, 만료 유효성을 순차적으로 검증한다.

토큰이 없거나 만료된 경우 401(TOKEN_INVALID)을 반환하며(만료 토큰은 DB에서 삭제), 유효한 경우 사용자 정보를 조회해 새 접근 토큰을 발급한다.

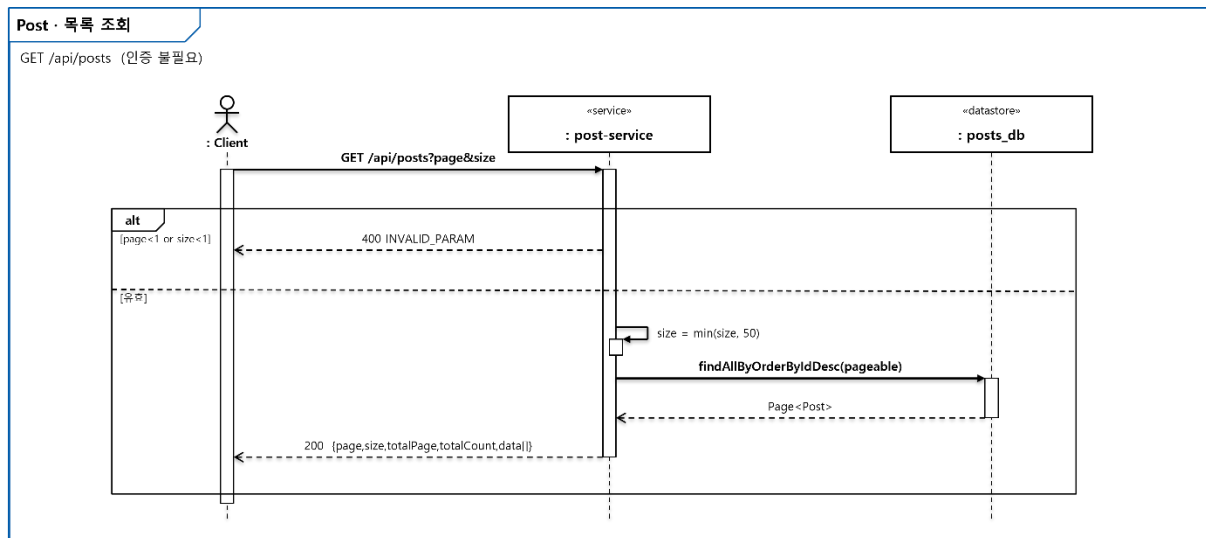


로그아웃 요청 시 Access Token의 유효성을 확인한 뒤, Refresh Token이 존재하면 이를 DB에서 삭제한다. 이어 응답에서 Refresh Token 쿠키를 즉시 만료시켜 세션을 종료한다. 서버 측 Refresh Token까지 폐기함으로써 로그아웃 이후 토큰 재사용을 차단한다.

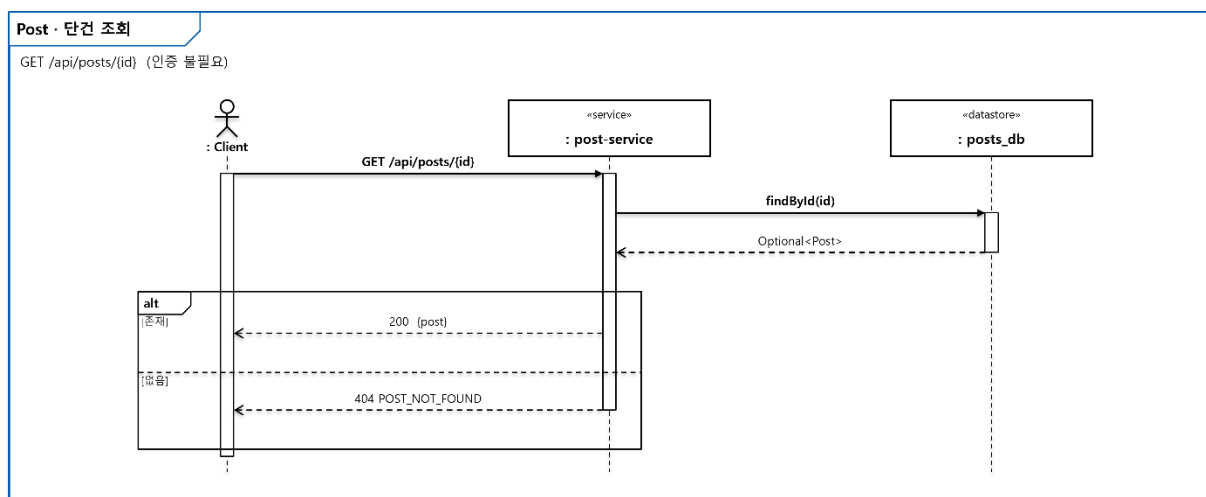


/internal/auth/validate는 외부에 노출되지 않는 내부 엔드포인트로, post-service, comment-service가 인가가 필요한 요청을 처리할 때 호출한다. auth-service는 전달받은 JWT의 서명과 만료를 검증하여 유효하면 사용자 식별 정보(userId, username)를, 무효하면 401(TOKEN_INVALID)을 반환한다.

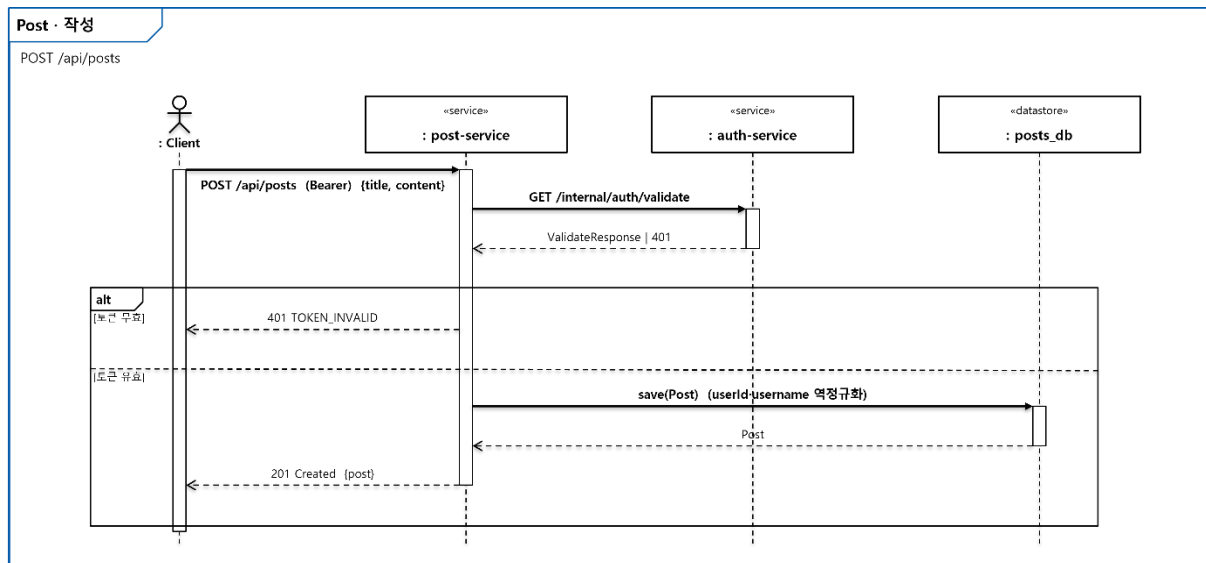
② 게시물 관리 (Post Service)



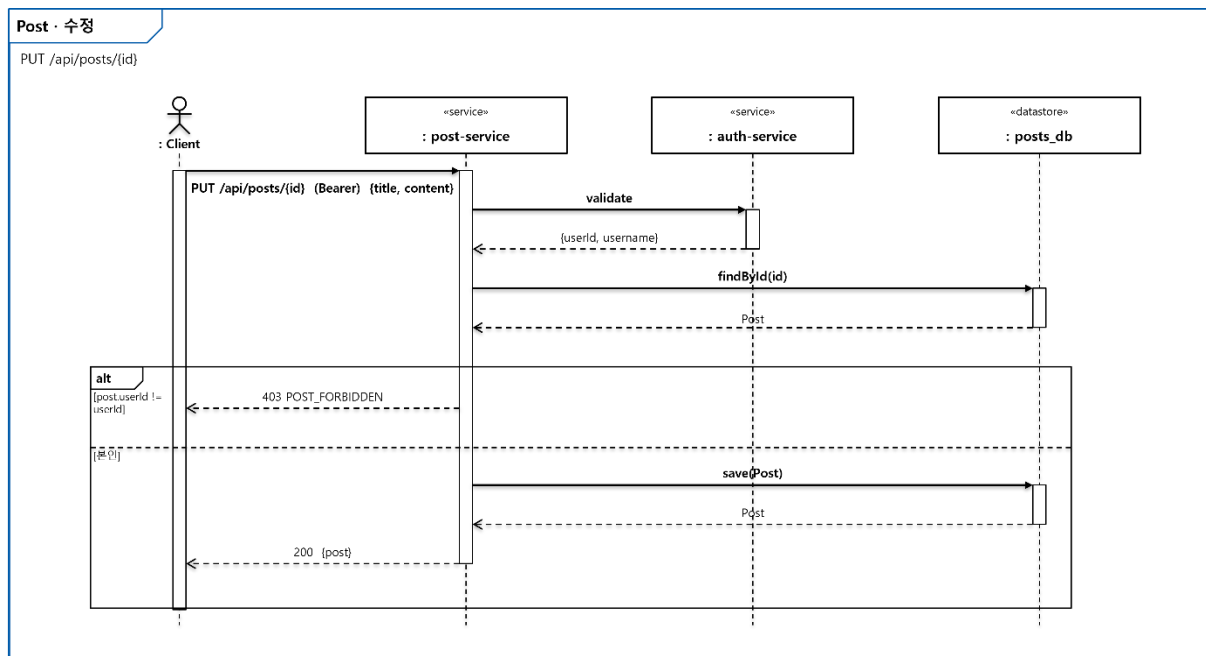
목록 조회는 인증 없이 가능하다. post-service는 page, size 파라미터의 유효성을 검증(잘못된 경우 400 INVALID_PARAM)한 뒤, id 내림차순으로 게시글을 페이지 단위로 조회하여 페이지 정보와 함께 반환한다.



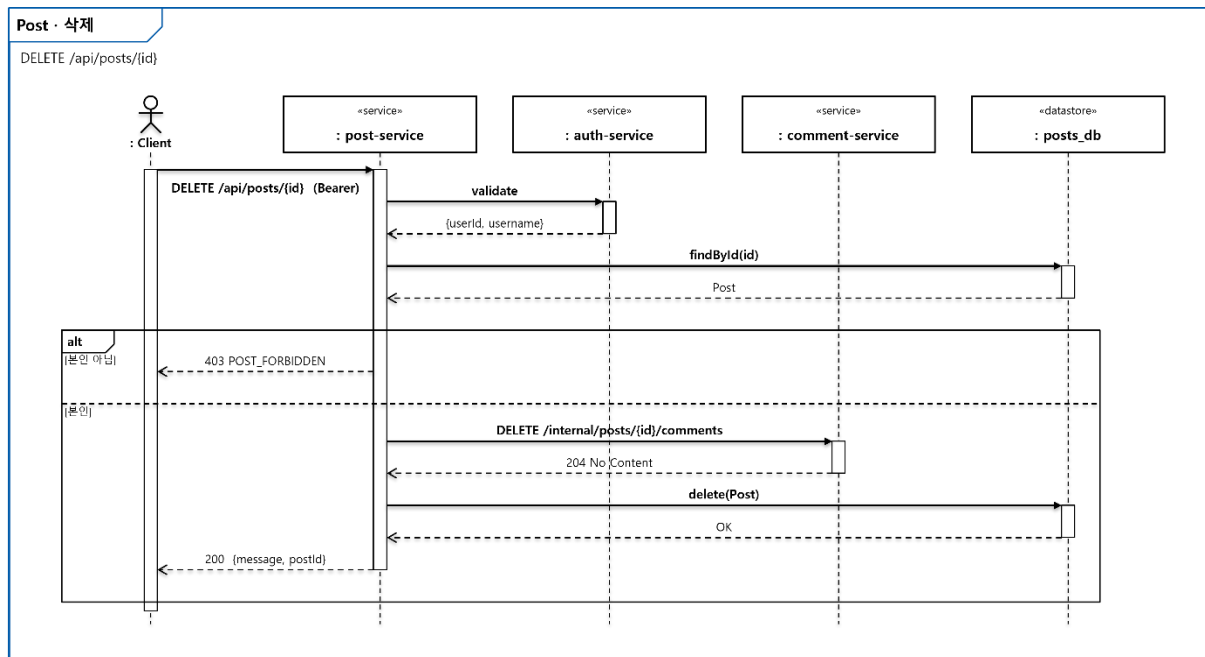
특정 게시글을 조회하며, 마찬가지로 인증이 필요 없다. 해당 id의 게시글이 존재하면 상세 정보를, 존재하지 않으면 404(POST_NOT_FOUND)를 반환한다.



인증이 필요한 기능으로, post-service는 요청의 Authorization 헤더를 auth-service의 validate로 전달해 사용자를 식별한다. 토큰이 무효면 401을 반환하고, 유효하면 조회 성능을 위해 작성자의 userid, username을 역정규화하여 게시글을 저장한 뒤 201 Created로 응답한다. 서비스 간 위임 검증이 처음 나타나는 흐름이다.

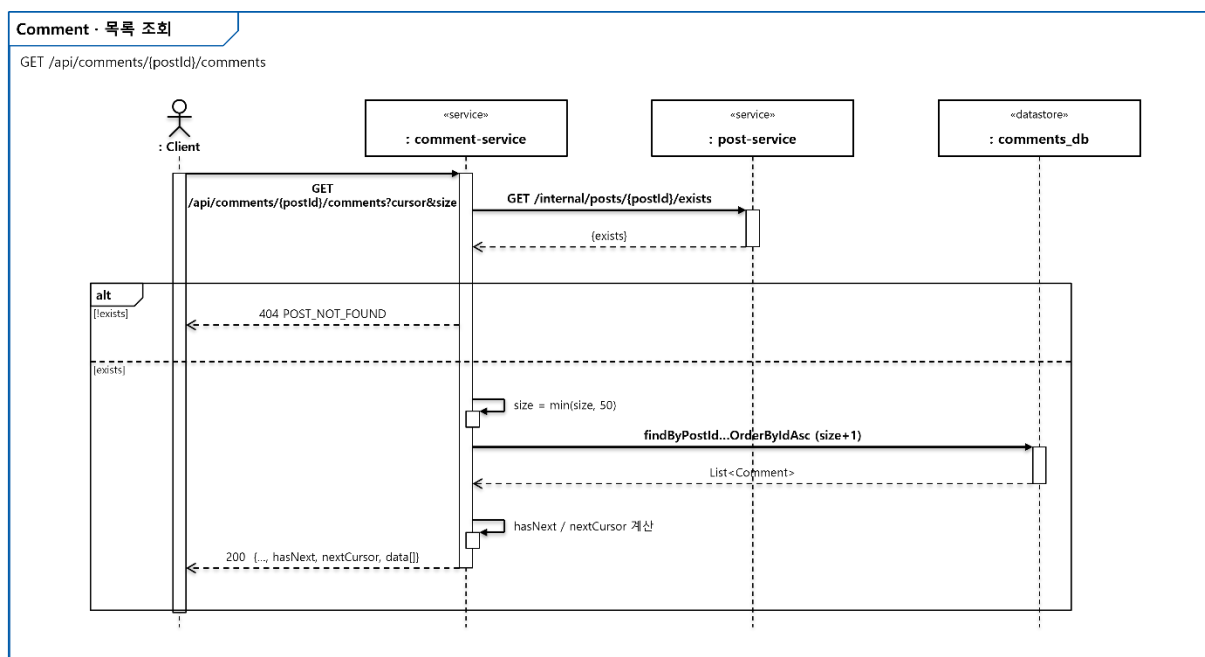


게시글 작성과 동일하게 validate로 사용자를 식별한 후 대상 게시글을 조회하고, 게시글의 작성자와 요청자가 일치하는지 소유권을 검사한다. 본인이 아니면 403(POST_FORBIDDEN)을 반환하며, 본인인 경우에만 내용을 수정하여 저장한다.



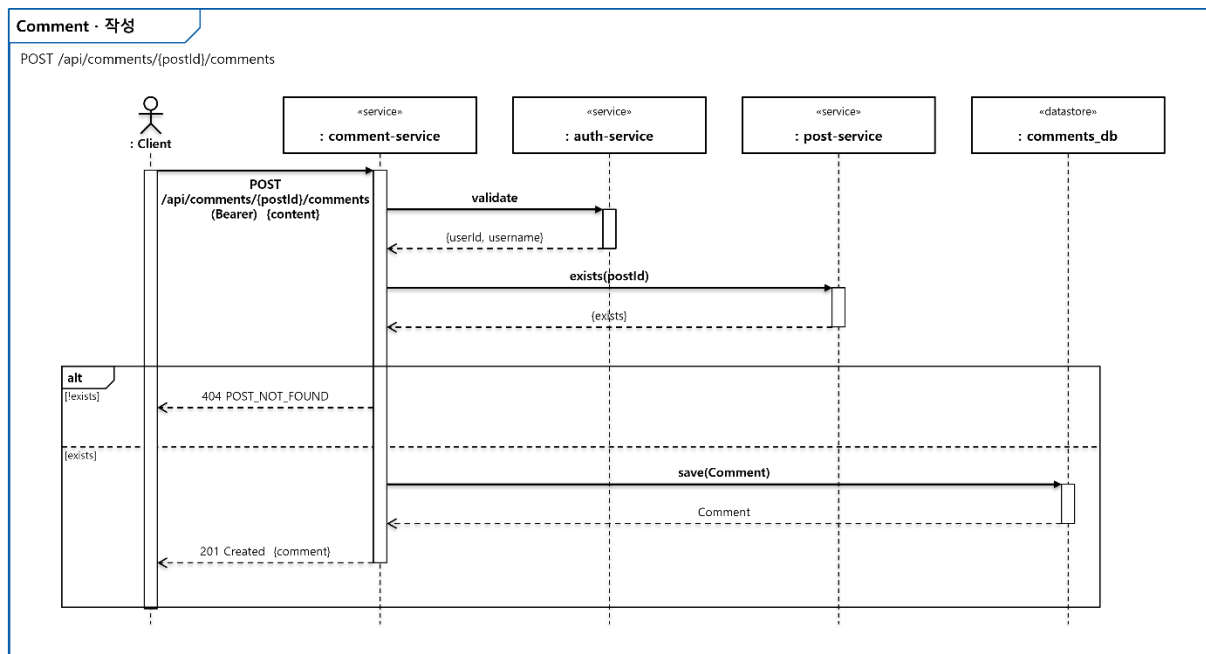
본인 확인 후, comment-service의 내부 엔드포인트(DELETE /internal/posts/{id}/comments)를 먼저 호출하여 해당 게시글의 댓글을 일괄 삭제한 뒤 게시글을 삭제한다. 댓글 삭제를 선행함으로써 comment-service 호출이 실패하면 게시글 삭제도 중단되도록 하여, 서비스 간 데이터 정합성(고아 댓글 방지)을 확보한다.

③ 댓글 관리 (Comment Service)

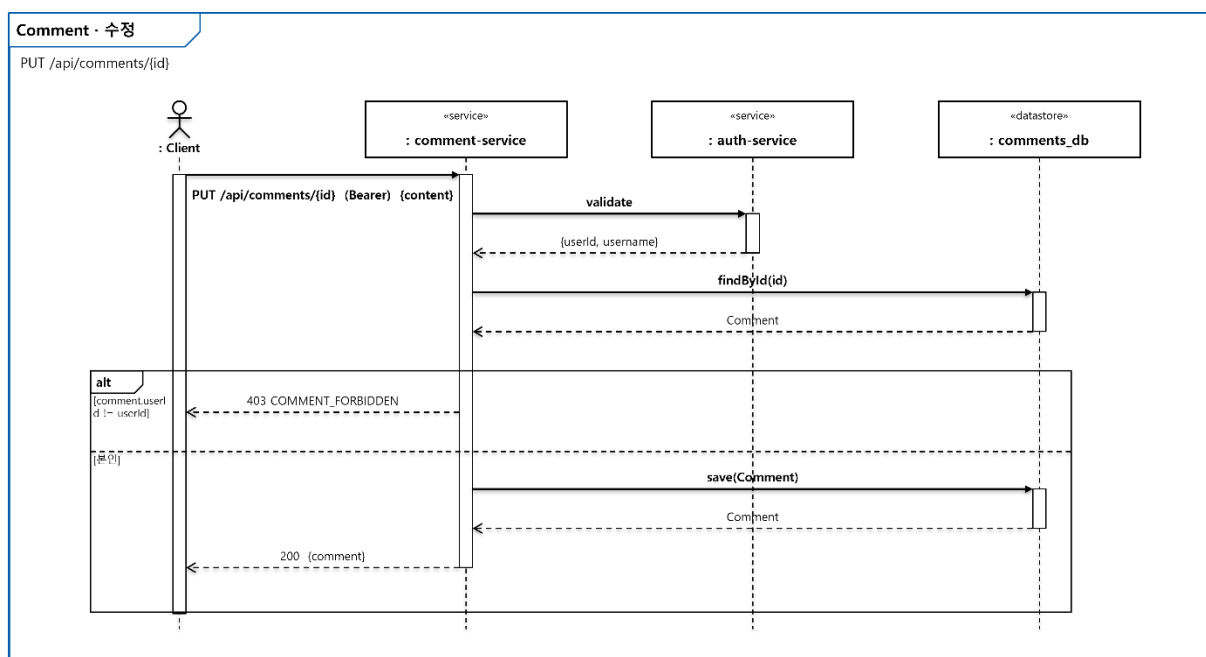


comment-service는 먼저 post-service의 exists를 호출하여 대상 게시글의 존재를 확인한다(없

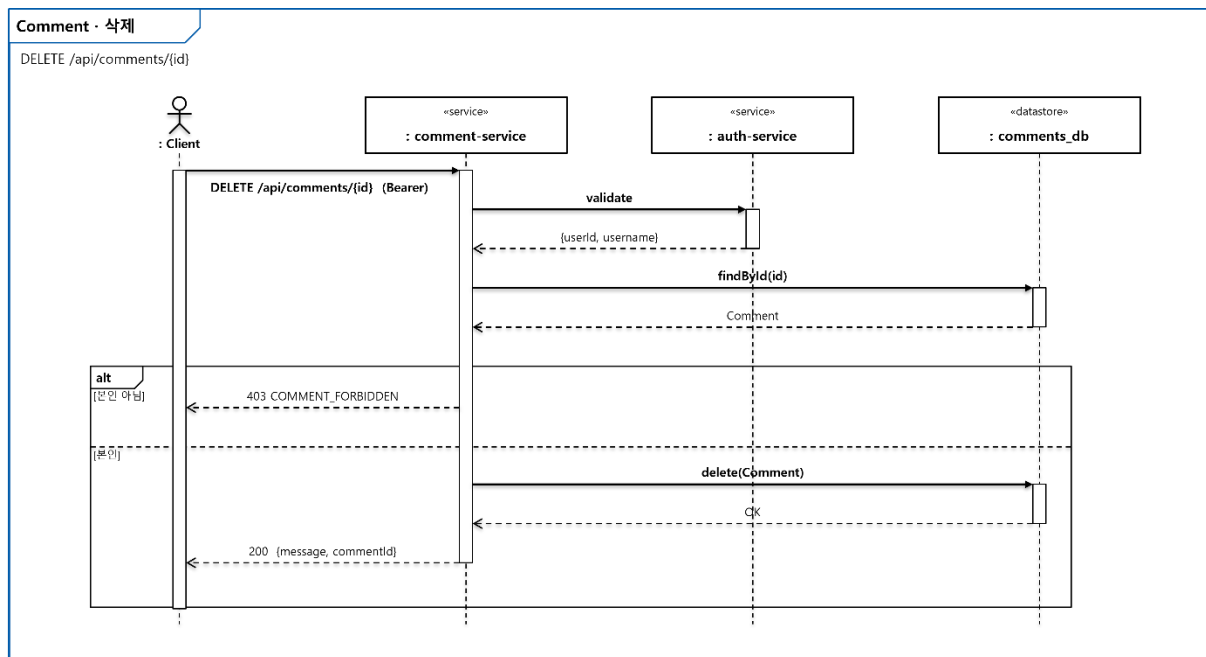
으면 404 POST_NOT_FOUND). 존재하면 size+1건을 조회해 다음 페이지 여부(hasNext)와 다음 커서(nextCursor)를 계산하는 cursor 기반 페이지네이션으로 댓글 목록을 반환한다.



댓글 작성은 인증이 필요하며, validate로 사용자를 식별한 뒤 exists로 게시물 존재를 확인하는 두 번의 서비스 간 호출이 발생한다. 게시물이 없으면 404를 반환하고, 정상인 경우 작성자 정보를 역정규화하여 댓글을 저장한 후 201 Created로 응답한다.



validate로 사용자를 식별한 뒤 대상 댓글을 조회하고 소유권을 검사한다. 본인이 아니면 403(COMMENT_FORBIDDEN)을 반환하며, 본인인 경우에만 내용을 수정하여 저장한다.



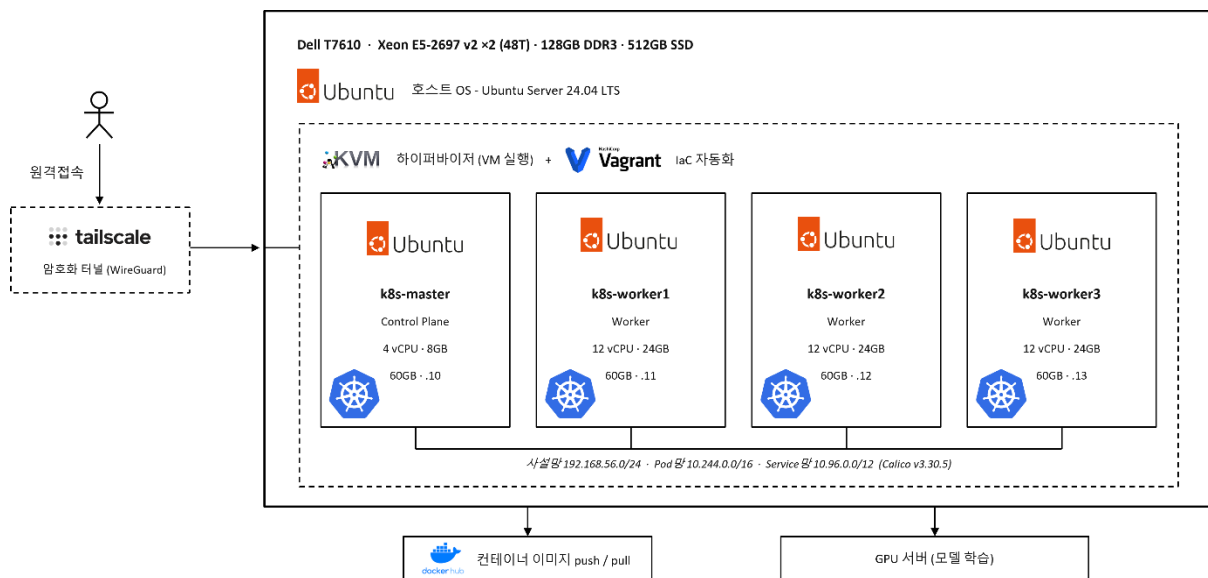
댓글 삭제도 수정과 동일하게 인증, 소유권 검사를 거친 뒤, 본인 확인 시 댓글을 삭제하고 삭제 결과를 반환한다.

세 서비스는 위와 같이 인증(Auth), 게시글(Post), 댓글(Comment) 기능을 제공하며, 인가 위임(validate), 존재 확인(exists), 연쇄 삭제 등 서비스 간 내부 호출을 통해 실제 MSA 환경의 east-west 트래픽을 발생시킨다. 이 실험환경은 이후 정상·공격 트래픽 수집과 서비스별 이상 탐지 모델 학습의 기반이 된다.

3-2. 쿠버네티스 클러스터 구성

실험 환경의 기반이 되는 Kubernetes 클러스터는 단일 물리 워크스테이션(Dell Precision T7610) 위에 가상머신 4대로 구축하였으며, 구축 과정을 Vagrant 기반 코드로 재현 가능하게 했다.

- 하드웨어 세팅 및 쿠버네티스 클러스터 구성



물리 장비는 두 개의 Intel Xeon E5-2697 v2(총 48 스레드), 128GB DDR3 메모리, 512GB SSD를 갖춘 Dell T7610 워크스테이션이다. 호스트 운영체제로 Ubuntu Server 24.04 LTS를 설치하고, 그 위에 가상화 스택(KVM/QEMU/libvirt)을 구성하였다. 여기서 KVM이 CPU-메모리를 가상화하는 하이퍼바이저 역할을 하고, Vagrant가 이 가상화 스택을 제어하여 가상머신을 생성한다. Vagrant를 이용한 이유는 재현성 때문이다. 인프라 구성을 Vagrantfile 코드를 통해 클러스터가 손상되어도 동일한 환경을 빠르게 복구할 수 있다.

```
vagrant@k8s-master:~$ kubectl get nodes -o wide
NAME          STATUS    ROLES          AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE          KERNEL-VERSION   CONTAINER-RUNTIME
k8s-master    Ready     control-plane   16d   v1.33.13  192.168.56.10 <none>         Ubuntu 24.04.2 LTS 6.8.0-64-generic  containerd://2.2.1
k8s-worker1   Ready     <none>          16d   v1.33.13  192.168.56.11 <none>         Ubuntu 24.04.2 LTS 6.8.0-64-generic  containerd://2.2.1
k8s-worker2   Ready     <none>          16d   v1.33.13  192.168.56.12 <none>         Ubuntu 24.04.2 LTS 6.8.0-64-generic  containerd://2.2.1
k8s-worker3   Ready     <none>          16d   v1.33.13  192.168.56.13 <none>         Ubuntu 24.04.2 LTS 6.8.0-64-generic  containerd://2.2.1
vagrant@k8s-master:~$
```

노드 이름	역할	vCPU	메모리	디스크	INTERNAL-IP
k8s-master	Control Plane	4	8GB	60GB	192.168.56.10
k8s-worker1	Worker	12	24GB	60GB	192.168.56.11
k8s-worker2	Worker	12	24GB	60GB	192.168.56.12
k8s-worker3	Worker	12	24GB	60GB	192.168.56.13

클러스터는 Master Node 1대와 Worker Node 3대로 구성했다. 48 스레드 중 40 vCPU를 가상 머신에 할당했는데, 대부분의 자원을 Worker Node에 배분한 것은 서비스 Pod와 각 Pod에 결합될 프록시 사이드카를 수용하기 위함이다. 메모리는 Master Node에 8GB, Worker Node에 24GB씩 할당하고, 디스크는 각 노드에 60GB씩을 할당했다.

3-3. 쿠버네티스 배포

이전에 구성한 MSA 애플리케이션을 Kubernetes 환경에서 배포할 수 있도록 manifest 구성을 정리하고, 추후 서비스 메시 기반 이상 탐지 구조를 실험할 수 있도록 Sidecar Proxy를 결합한 manifest 구성도 추가하였다. 먼저 Auth, Post, Comment, Frontend 서비스에 대해 각각 Deployment와 Service manifest를 구성하였다. 각 서비스는 Dockerfile을 기반으로 컨테이너 이미지를 빌드한 뒤 Docker Hub에 업로드하였으며, Deployment manifest에서는 해당 이미지를 참조하여 Pod가 실행되도록 설정하였다.

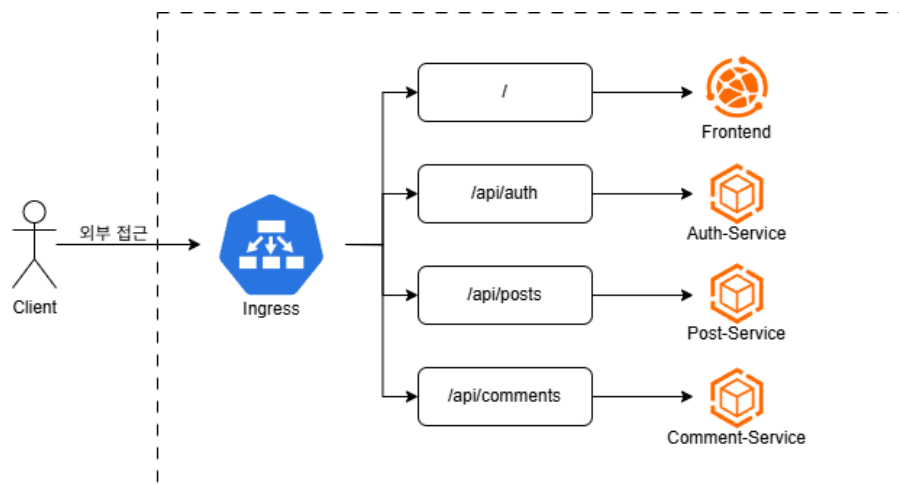
각 백엔드(Auth, Post, Comment) 서비스는 독립적인 Spring Boot 애플리케이션으로 실행되며, Frontend는 별도의 컨테이너로 배포되도록 구성하였다. 또한 서비스 간 통신에 필요한 주소, 포트, 환경 변수는 ConfigMap과 Secret을 통해 관리하도록 하였다. DB는 MySQL을 Kubernetes 내부에서 실행하기 위해 StatefulSet, Service, PersistentVolumeClaim을 사용하였다. 이를 통해 Pod가 재시작되더라도 데이터가 유지될 수 있도록 하고, 각 백엔드 서비스가 동일한 엔드포인트를 통해 MySQL에 접근할 수 있도록 설정하였다.

구성 대상	사용 리소스	주요 역할
Auth-Service	Deployment, Service	Auth 백엔드 실행 및 내부 통신 제공
Post-Service	Deployment, Service	Post 백엔드 실행 및 내부 통신 제공
Comment-Service	Deployment, Service	Comment 백엔드 실행 및 내부 통신 제공
Frontend	Deployment, Service	사용자 화면 제공
DB(MySQL)	StatefulSet, Service, PVC	DB 실행 및 데이터 영속성 제공
공통 설정	ConfigMap, Secret	환경 변수 및 민감 정보 관리

- Deployment: 애플리케이션 Pod의 생성, 복제 수, 업데이트 방식 관리
- Service: Pod에 접근하기 위한 고정된 네트워크 엔드포인트 제공
- PersistentVolumeChain: Pod가 사용할 영구 저장 공간을 요청

추가로 외부 접근을 위해 Frontend 서비스와 API 경로를 연결하는 Ingress manifest를 구성하였다. Ingress는 하나의 주소로 들어온 요청을 경로에 따라 프론트엔드와 각 백엔드 API 서비스로 전달하며, 이를 통해 사용자는 하나의 진입점으로 전체 MSA 애플리케이션을 이용할 수 있다.

- Ingress를 통한 MSA 요청 라우팅



이후 실제 배포는 Kubernetes 리소스 간 의존성을 고려하여 순차적으로 진행하였다. 먼저 namespace를 생성하고, 서비스에서 공통으로 사용하는 ConfigMap과 Secret을 적용하였다. 다음으로 MySQL 실행을 위한 PersistentVolumeClaim, Service, StatefulSet을 적용하여 데이터베이스가 먼저 실행될 수 있도록 구성하였다. DB 구성이 완료된 후 Auth, Post, Comment 백엔드 서비스를 Deployment + Service를 통해 순서대로 배포하고, 각 서비스의 Pod와 Service가 정상적으로 생성되는지 확인하였다. 이후 Frontend 서비스를 배포한 뒤 Ingress를 적용하여 외부 요청이 프론트엔드와 각 백엔드 API로 전달될 수 있도록 구성하였다.

마지막으로 일반 배포 manifest와 별도로 Sidecar Proxy가 포함된 Deployment manifest를 구성하였다. 이를 통해 기존 애플리케이션 컨테이너와 Proxy 컨테이너가 하나의 Pod 안에서 함께 실행될 수 있도록 하였으며, 추후 서비스 간 트래픽을 Proxy를 통해 관찰하고 이상 탐지 기능과 연계할 수 있는 기반을 마련하였다.

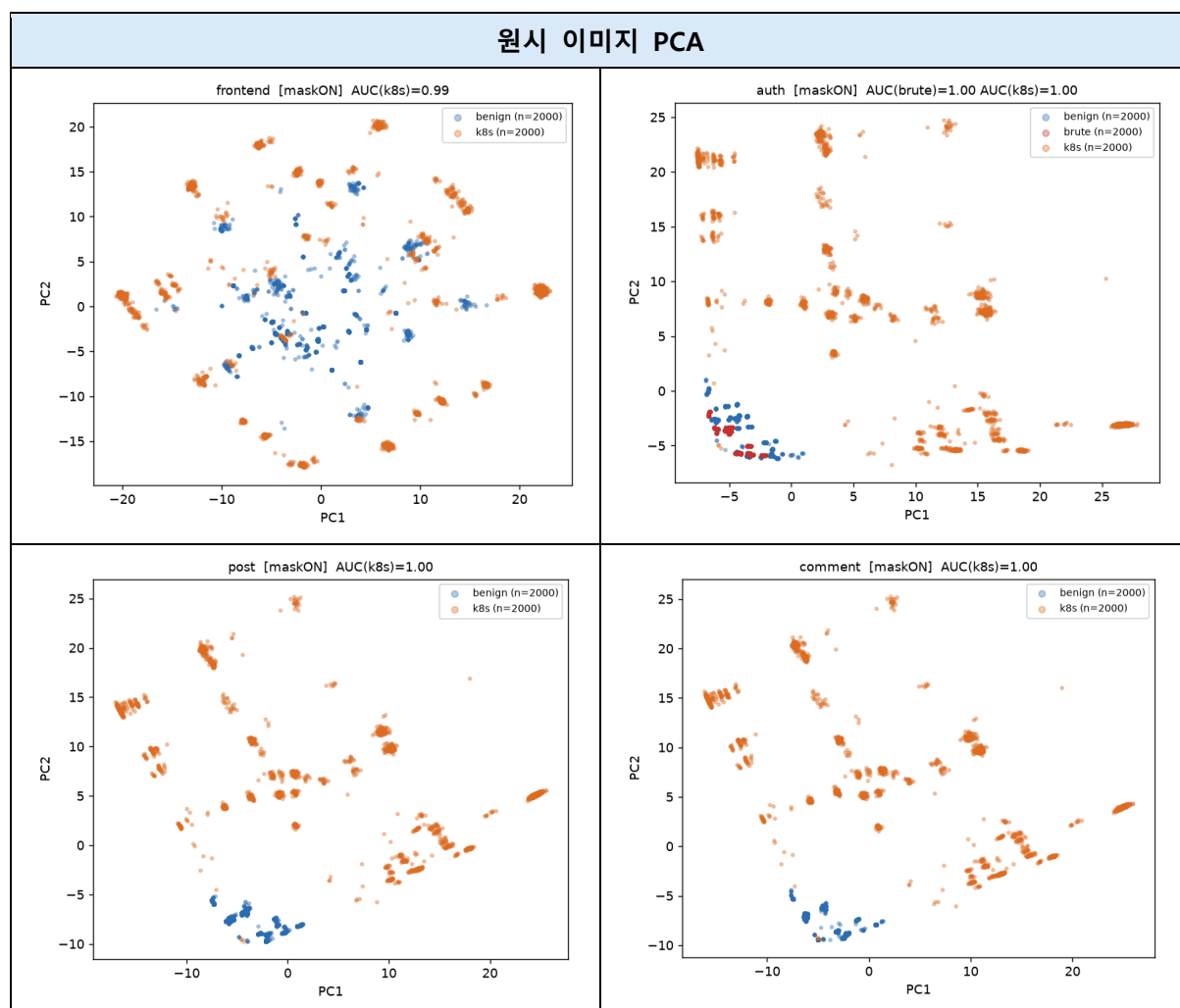
3-4. 트래픽 데이터 수집

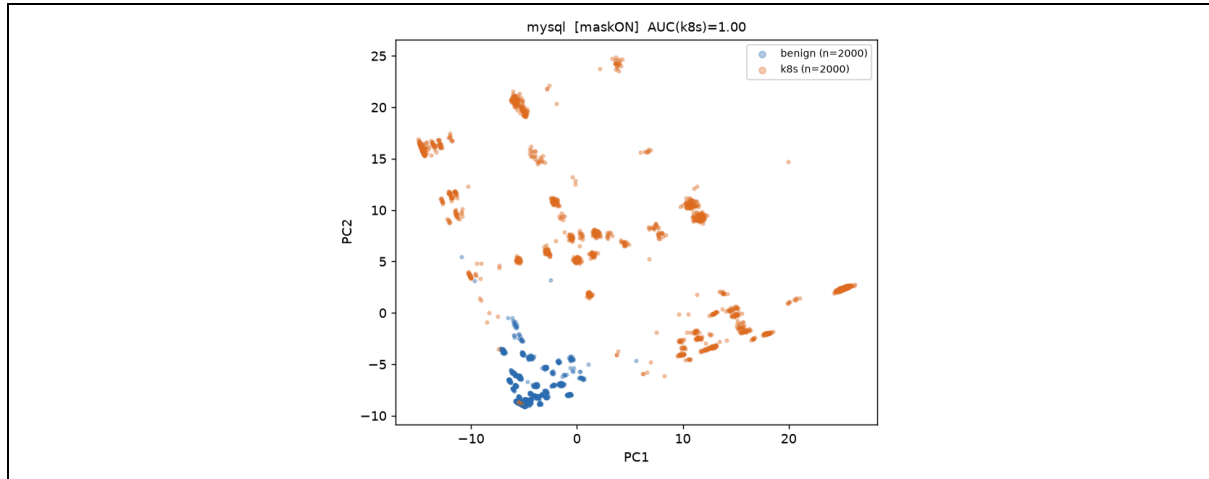
위험 모델은 침해된 pod의 K8s 횡적 이동(east-west)으로, 초기 침입을 가정하고 침해 후 pod가 내보내는 트래픽을 탐지 대상으로 삼는다. 수집 대상은 자체 클러스터의 게시판 MSA(auth, post, comment, frontend, mysql)이다.

정상 트래픽은 사용자의 north-south 요청뿐 아니라 그로 인해 백엔드가 서로를 호출하는 정상 east-west까지 담아 서비스 간 호출 자체를 공격으로 오인하는 문제를 방지하였다. 또한 정상 사용 중에도 발생하는 4xx가 공격 신호로 오인하지 않게 했고, 수집이 끝난 후에는 트래픽 생성 직전에 생성한 DB 스냅샷을 복원하여 시스템 상태를 유지했다.

공격은 클러스터 내부 공격자 pod 안에서 실행하였다. 시나리오는 클러스터 정보 열거(T1613, T1528), 리소스 조작(T1609), 능동적 스캐닝과 브루트포스 공격(T1595, T1110) 총 3가지로 구성하였다. 캡처는 각 pod의 네트워크 네임스페이스 내 eth0(Ethernet 프레임)에서 수행하고, 스냅 길이를 0으로 두어 payload를 절단 없이 담으며, 세션 단위 분석을 위해 세션당 다섯 패킷 이상, 요청 및 응답 양방향을 확보하였다.

수집한 트래픽 자체의 분리 가능성을 지도학습으로 확인해 수집된 트래픽 데이터가 구분 가능한 신호를 충분히 포함하고 있는지 먼저 검증하였다. 아래는 마스킹된 원시 이미지 픽셀을 PCA로 투영한 것으로, 원시 바이트 관점의 분리도 최적을 보여준다. Post, Comment 백엔드는 정상 단일 클러스터로 뭉치고 공격(K8s API egress)이 전역에 흩어져 깔끔히 분리되는 반면, Auth 백엔드는 브루트포스가 정상 로그인과 바이트가 유사해 2D 산점에서는 정상 위에 포개지나, top-50 성분의 지도 분류로 구분할 수 있어 원시에도 신호가 존재함을 확인할 수 있다. frontend는 정상 트래픽이 매우 다양해 원시 바이트로는 가장 혼재된다. 지도 학습 결과 모든 서비스에서 AUC가 0.99 이상으로 매우 낙관적인 결과를 보였다.





3-5. 모델 평가

수집한 pcap은 로컬 GPU 환경에서 세션 이미지로 변환한다. 각 패킷을 19바이트 헤더와 1460바이트 payload를 합한 1479바이트로 표현하되 환경 의존적인 IP와 포트는 제외해 학습 및 추론 표현을 일치시키고, 5-tuple 세션에 슬라이딩 윈도우 $w=5$ 로 5×1479 이미지를 만든다. 내부 검증은 세션 단위로 분할해 근사 중복 누수를 막고, 전송계층 필드에 해당하는 window, seq, ack를 마스킹해 캡처 환경 confound를 제거했다.

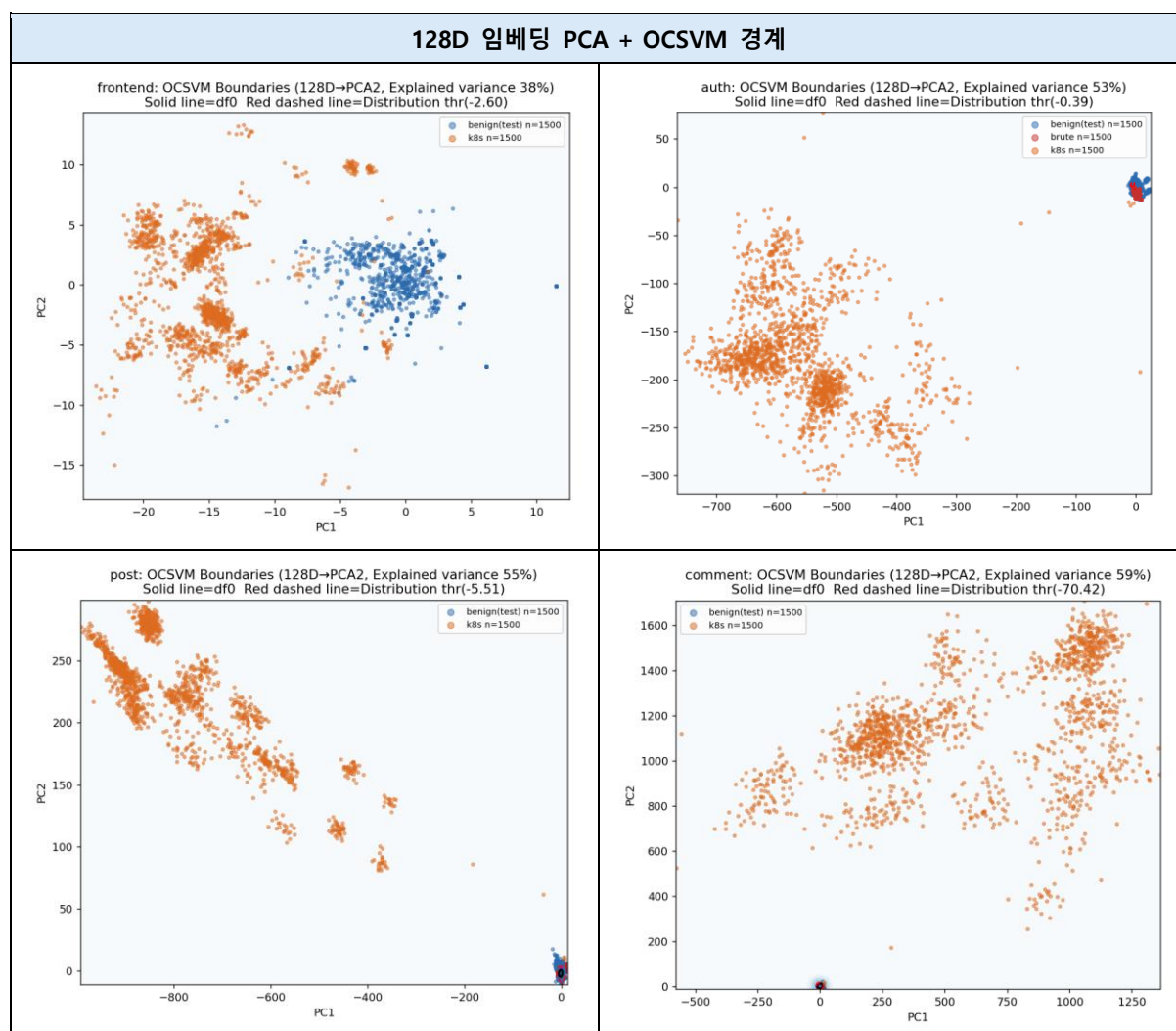
모델은 표현 학습을 담당하는 KD-CNN과 이상 점수화를 담당하는 OCSVM을 분리 결합한다. teacher는 NT-Xent 대조학습 shallow 인코더(314.69K)를 사용하고, 이를 지식증류로 경량 student로 압축한 뒤 정상만으로 OCSVM을 학습한다. 학생 모델을 확정하기 위해 서비스 별로 동일 teacher에 대해 1×8 , 1×16 , 2×8 , 2×16 , 2×32 다섯 종을 증류한 결과를 비교했으며, 2×8 (params = 5.69K)이 크기 대비 최적점을 보이는 것을 확인했다. 평가는 학습에 전혀 사용하지 않은 별도 시점의 정상 및 공격 트래픽으로 평가했다.

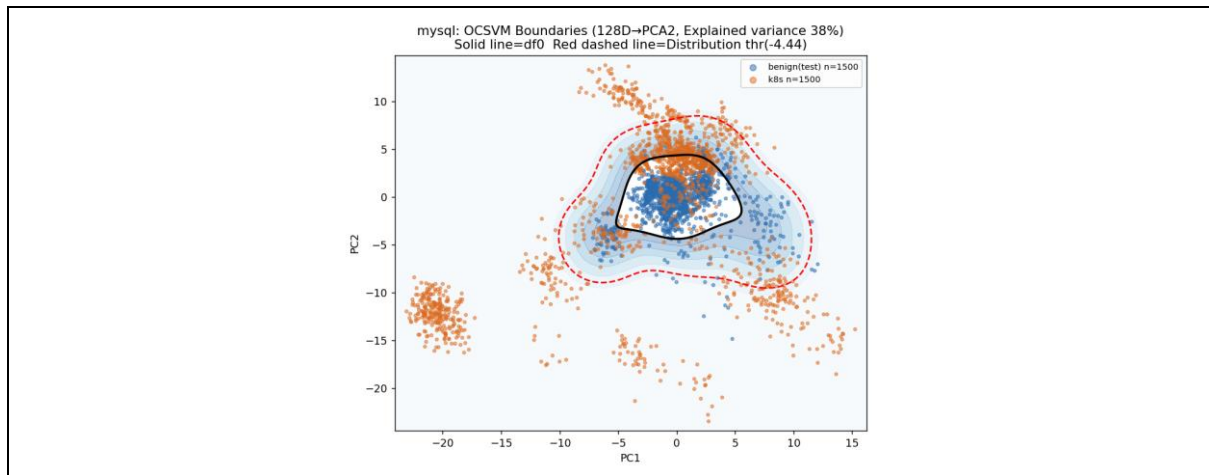
Service	Arch	Test benign	Test attack	F1	ROC-AUC	Recall
auth	2×8	14,309	30,000	0.9658	0.9428	1.000 (brute=0.971)
post	2×8	15,000	15,000	0.9686	0.9999	1.000
comment	2×8	15,000	15,000	0.9944	1.0000	1.000
frontend	2×8	4,499	15,000	0.9950	0.9994	0.997
mysql	2×8	15,000	15,000	0.9933	0.9992	1.000

전 서비스에서 Recall은 0.97~1.00, ROC-AUC는 0.925~1.00, F1은 0.966~0.995를 기록하였다.

이러한 낙관적인 결과가 데이터 누수에 기인한 것이 아님을 확인하기 위해 각 서비스 별 train과 test의 행 해시 교집합이 전 서비스에서 0%로 나타나는지 검증하였다. 대부분의 K8s 이탈 공격이 침해된 pod가 :443 TLS로 API를 호출하여 정상 트래픽(8080 HTTP)과 프로토콜 및 목적지가 다른 문제였기 때문에 탐지 성능이 높았다.

아래 그림은 학습된 128차원 임베딩을 PCA로 투영하고 실제 OCSVM 결정경계를 겹친 것으로, 배포 모델이 만들어낸 분리를 보여준다. 그림에서 검은 실선(df0)은 OCSVM의 결정함수 값이 0이 되는 경계로 sklearn 기본 컷에 해당하며, 그 안쪽(df>0)을 정상 영역으로 본다. 빨간 점선(distribution thr)은 학습 정상의 점수 분포로부터 목표 오염률인 1%를 만족하도록 보정한 배포 임계값으로, 실제 판정은 이 선을 기준으로 결정함수 값이 임계값보다 낮으면(df<thr) 이상으로 분류한다. 해당 임계값은 auth에서 0.39, comment에서 70.42로 약 178배 차이를 보였다. 이는 스택(Spring+JSON+MySQL)이 동질적임에도 서비스별 정상 행동이 상이하여 결정 기하가 다르게 형성되기 때문으로, 서비스별 모델 채택의 근거가 된다.





특히 frontend 서비스에서, 원시 바이트 수준에서는 섞여 보이던 트래픽을 인코더가 두 그룹으로 분류하였다. 이는 선행 연구에서 제시했던 원시 바이트 상의 정상/공격 분포가 겹치는 문제를 임베딩으로 분리해낸 사례에 해당한다. post와 comment는 공격이 K8s API egress(TLS)로 정상 HTTP와 완전히 이질적이었기 때문에 임베딩 공간에서 크게 멀어졌고, auth는 브루트포스 공격이 임베딩에서도 정상에 근접하였기 때문에 ROC가 다른 서비스에 비해 Recall이 0.971, ROC-AUC가 0.943으로 가장 낮았다.

4. 남은 개발 단계 설계 상세화

4-1. remote attack과 escape 시나리오의 추가로 공격 기법 커버리지 확장

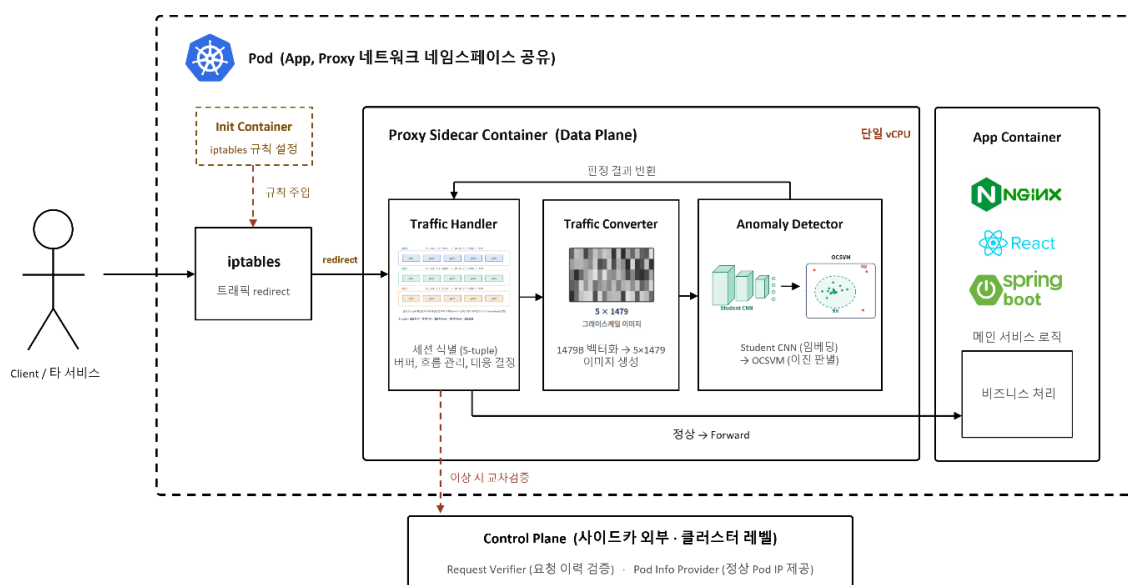
현재 커버리지(enum, manipulate의 K8s egress와 auth 브루트포스)에 remote attack과 escape(컨테이너 탈출) 시나리오를 추가하여, 클러스터 정찰과 조작, 원격 접근으로 이어지는 실제 공격도 탐지 가능한 범위인지 확인한다. Remote attack은 정상과 동일한 프로토콜과 엔드포인트를 사용하는 hard case로 설계하여, TLS egress처럼 이질적이기 때문에 쉽게 걸리는 공격이 아니라 정상에 섞여 드는 어려운 공격까지 탐지할 수 있는지를 입증하는 것이 목표다. escape는 침해된 pod가 컨테이너 경계를 넘어 호스트와 노드 권한을 노리는 침해의 최종 단계로, kubelet, K8s API, 호스트 자원 접근 등 정상 앱 트래픽과 목적지 및 행위가 다른 패턴을 가진다.

공격 기법 커버리지 확장을 통해 특정 기법에만 통하는 것이 아니라는 일반화 근거를 강화하고, auth 브루트포스에 이은 새로운 hard case의 발견과 탐지로서 이상탐지 방법의 실질적 변별력을 재검증한다.

4-2. 반복 실행으로 통계적 변동성 보강

학습 및 평가를 여러 랜덤 시드로 반복해 teacher와 student의 가중치를 초기화하고, train/val 세션 분할의 우연성이 성능에 주는 영향을 측정하여 서비스별 F1 Score와 ROC-AUC를 단일 값이 아닌 평균과 표준편차 값을 이용한 신뢰구간으로 보고한다. 또한 서로 다른 시점 및 부하와 다른 세션으로 정상/공격 트래픽을 여러 차례 재수집하여 캡처 시점에 따른 우연성에 대한 영향을 줄인 데이터셋으로 평가한다. 위와 같은 방법으로 모델 초기화 변동과 캡처 데이터 변동을 분해해, 안정적으로 높은 성능을 보이는 것을 통계적으로 입증한다.

4-3. 프록시 사이드카 및 이상 탐지 모듈 탑재



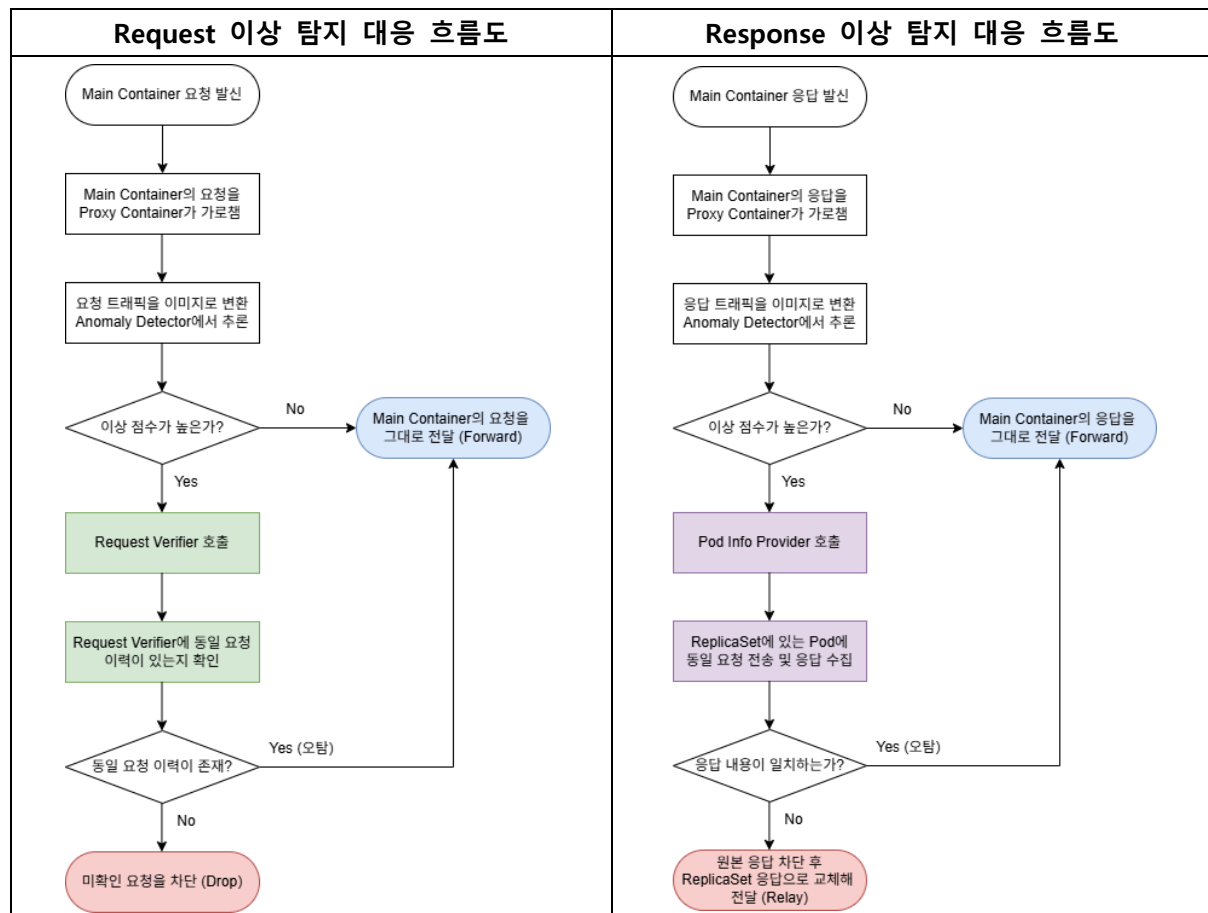
이상 탐지 모듈은 각 서비스 Pod에 사이드카 프록시 컨테이너로 결합한다. 프록시는 메인 애플리케이션 컨테이너와 동일한 Pod 내에서 네트워크 네임스페이스를 공유하며, Init Container가 iptables 규칙을 설정하여 Pod의 모든 인, 아웃바운드 트래픽을 프록시 포트로 우회(redirect)시킨다. 이 구조 덕분에 애플리케이션 코드를 수정하지 않고 트래픽을 가로챌 수 있다.

프록시 내부는 인터셉트된 트래픽을 이상 판정까지 처리하는 파이프라인으로 설계한다. 모델 탑재는 학습 단계에서 산출한 Student CNN(.pt)과 OCSVM(.pkl)을 프록시 Docker 이미지에 포함하는 방식으로 구현한다. 실시간 추론은 트래픽 이미지를 Student CNN 인코더에 입력해 임베딩을 얻고, 이를 OCSVM으로 이진 판별하여 결과를 트래픽 처리부로 반환하는 흐름이다. 프록시 컨테이너는 단일 vCPU로 자원을 제한한다.

이상탐지 지연은 단일 vCPU 조건에서 이미지 1장당 처리 시간으로 측정한다. 파이프라인은 패킷 파싱과 이미지화, Student CNN 임베딩, OCSVM 판정의 세 단계로 분해하여 각 구간 소요와 병목을 식별한다. 측정은 워밍업 구간을 제외하고 충분한 반복 표본에 대해 수행하며, 평균 처리량(req/s)을 산출하는 것을 목표로 한다. 산출값은 선행 연구에서 제시한 기준인 이미지당 추론 지연 시간 평균 0.518ms, 초당 처리량 600 req/s와 대조하여¹, 배포 모델(2x8, 5.69K)이 실시간 라인 탐지에 요구되는 지연 예산을 충족하는지를 검증한다.

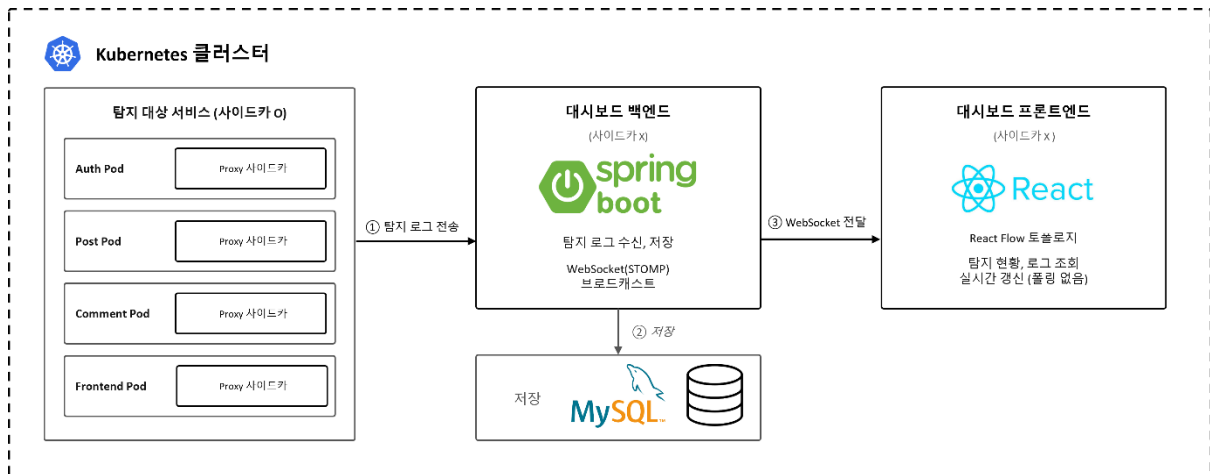
4-4. 트래픽 후속 처리 및 control plane 교차 검증

이상 탐지 결과에 따른 후속 처리는 Forward, Drop, Relay의 세 가지 대응으로 설계한다. 정상으로 판별되면 트래픽을 그대로 전달(Forward)하고, 이상으로 판별되면 마지막 패킷의 종류에 따라 서로 다른 검증을 거쳐 대응을 결정한다. 이 검증은 프록시 단독으로 완결되지 않고 Control Plane과의 교차 검증을 필요로 한다.

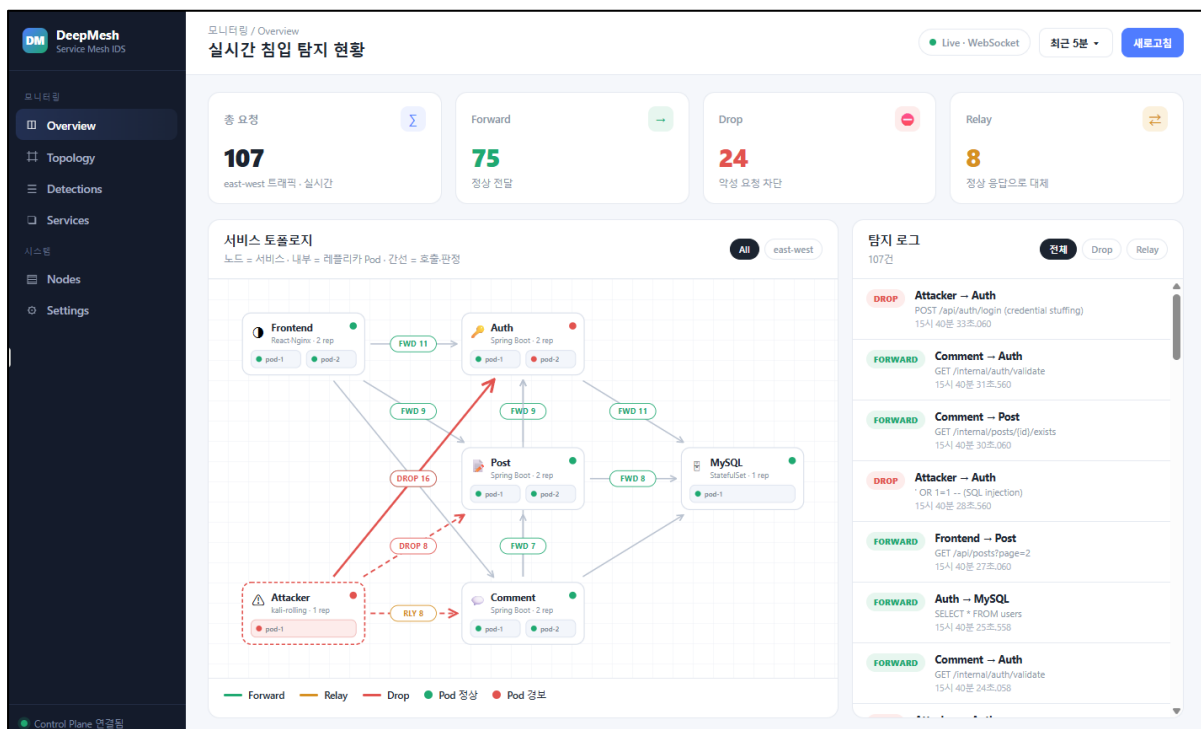


이 대응 체계는 동일하게 구성된 Pod는 동일 요청에 동일 응답을 생성한다는 replica 일관성 가정에 기반한다. 마지막 패킷이 요청인 경우, Request Verifier가 동일 ReplicaSet 내 다른 Pod에 같은 요청 이력이 존재하는지 교차 검증한다. 이력이 존재하면 정상 트래픽으로 간주해 Forward하고, 존재하지 않으면 해당 Pod에만 발생한 비정상 요청으로 판단해 Drop한다. 마지막 패킷이 응답인 경우, Pod Info Provider가 동일 ReplicaSet의 정상 Pod로부터 참조 응답을 수집한다. 참조 응답과 일치하면 Forward하고, 다르면 침해된 응답으로 판단해 정상 Pod의 응답으로 대체(Relay)함으로써 서비스 가용성을 유지한다.

4-4. 실시간 모니터링 대시보드

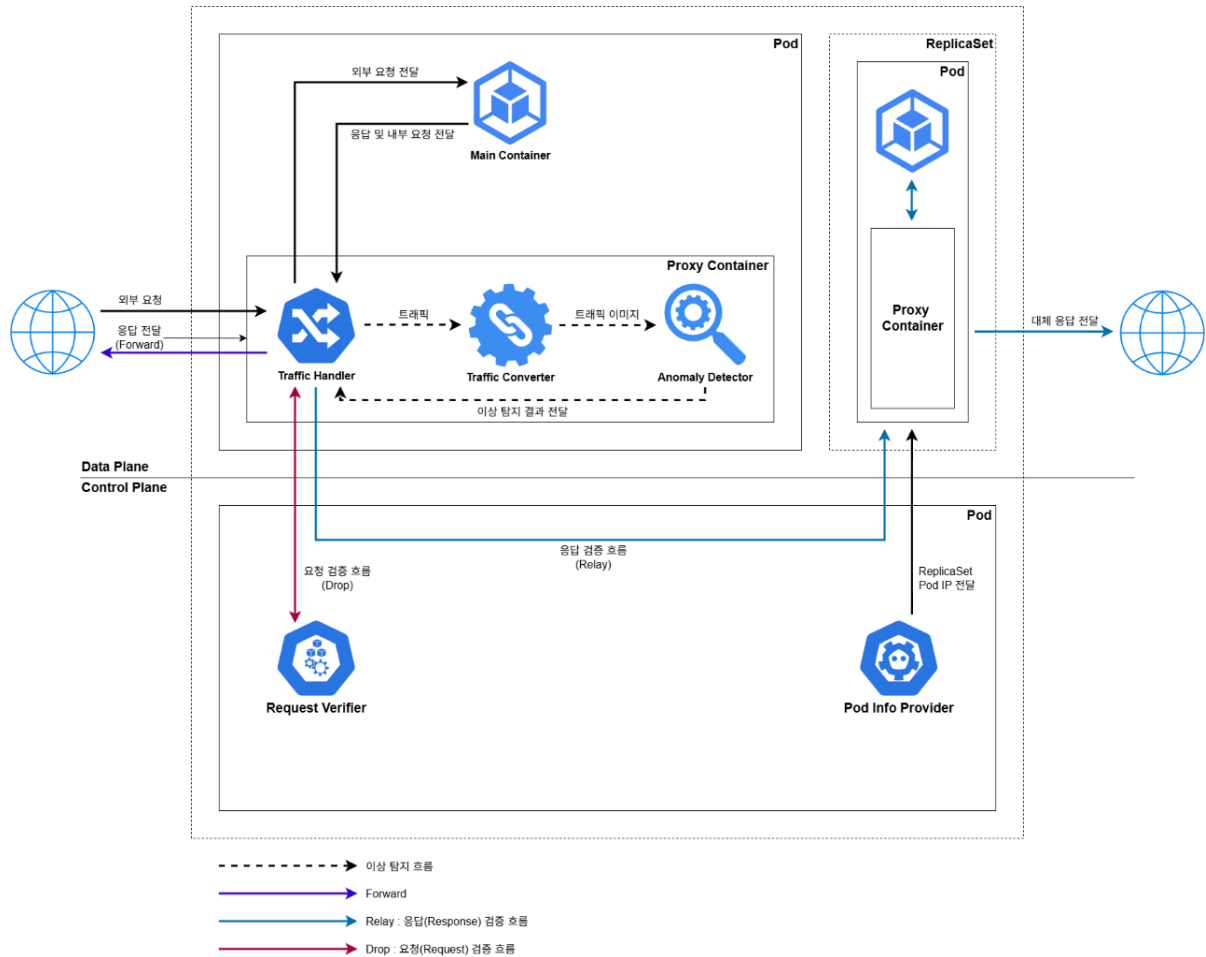


대시보드는 탐지 현황과 서비스 간 트래픽 흐름을 실시간으로 시각화하는 관리자용 도구로 설계한다. 데이터 흐름은 프록시가 탐지 이벤트를 대시보드 백엔드로 전송하는 것에서 시작한다. 각 프록시가 Forward/Drop/Relay 결과를 탐지 로그로 기록하면, 대시보드 백엔드가 이를 수신해 WebSocket으로 프론트엔드에 전달한다. 이 방식은 폴링 없이 실시간 갱신을 가능하게 한다.



위 그림은 관리자 대시보드에 대한 목업 페이지이다. 프론트엔드는 React 기반으로, 서비스 간 호출 관계를 React Flow 토폴로지 그래프로 표현하고, 탐지 현황과 로그 조회를 함께 제공한다. 대시보드 백엔드, 프론트엔드와 MySQL에는 프록시 사이드카를 부착하지 않는데, 이들은 탐지 대상 트래픽을 생성하는 서비스가 아니라 탐지 결과를 다루는 구성요소이기 때문이다.

5. 전체 시스템 통합 구조



전체 시스템 통합은 개별적으로 개발, 검증된 모듈들을 연결하고 end to end 흐름을 검증하는 단계로 진행한다. 통합해야 할 접합부는 크게 네 가지이다. 프록시와 제어 평면의 접합(Drop 시 Request Verifier, Relay 시 Pod Info Provider의 실제 호출), 모델과 프록시의 접합(GPU 서버에서 학습한 모델이 단일 vCPU 프록시에서 추론), 데이터 평면과 대시보드의 접합(탐지 이벤트가 WebSocket을 거쳐 토폴로지에 반영), 그리고 end to end 시나리오 검증(공격 스크립트 실행 → 탐지 → 교차 검증 → 대응 → 대시보드 반영)이다.

6. 구성원별 개발 진척도 및 향후 추진 계획

6-1. 중간보고서 작성 시점 개발 진척도

아래는 중간보고서 작성 시점까지 진척도(1단계 + 트래픽 데이터 수집)에 대해 각자 담당한 역할을 표로 정리한 것이다.

팀원	역할	세부 내용
신의철	MSA Back-end K8s 클러스터 구축	ERD 작성, endpoint 설계 Auth Service 개발 (JWT) K8s 클러스터 구축
정의진	MSA Front-end K8s MSA 프로젝트 배포	React + Nginx 기반 Frontend 개발 K8s 클러스터에 실험환경 배포
이시하	MSA Back-end 트래픽 데이터 수집 준비	endpoint 설계 Post Service, Comment Service 개발 트래픽 데이터 설계 및 수집 파이프라인 구축

6-2. 중간보고서 이후 향후 추진 계획

착수보고서에서 계획했던 일정을 그대로 따라간다. 아래 표와 같이 업무를 분장하여 각각 담당 파트를 구분하지만, 2단계부터는 개발 모듈이 유기적으로 영향을 주는 만큼 담당 역할에만 구속되지 않고 협업하여 개발을 진행한다.

- 2단계: 데이터 수집 및 모델 구현 (7월)

팀원	역할	세부 내용
신의철	Control Plane 개발	Request Verifier, Pod Info Provider 개발
정의진	Proxy Container 개발	iptables, Traffic Handler, Traffic Converter 개발 이상탐지모듈 결합, end to end 테스트
이시하	KD-CNN + OCSVM 개발 이상탐지모듈 평가	Teacher, Student 모델 학습 (Knowledge Distillation) OCSVM 이진 분류 학습 공격데이터를 이용한 성능 평가

• 3단계: 모니터링 대시보드 개발 및 시스템 통합 (8월)

팀원	역할	세부 내용
신의철	Dashboard Front-end	실시간 topology 그래프 개발 실시간 탐지 현황 및 이력 조회
정의진	Dashboard Back-end	ERD 작성, endpoint 설계 및 개발 WebSocket을 이용한 실시간 브로드캐스트 K8s 대시보드 배포
이시하	Dashboard Back-end	endpoint 설계 및 개발 실시간 탐지 로그 데이터 수집 (Push 또는 Pull 방식) 실시간 탐지 현황 분석

아래는 향후 추진 계획에 대한 주간 계획표이다.

수행내용	담당	5월		6월				7월					8월				9월		
		3	4	1	2	3	4	1	2	3	4	5	1	2	3	4	1	2	
1단계: k8s 클러스터 구축 및 MSA 실험환경 개발																			
FE, Nginx	정의진				기말고사														
Post, Comment Service	이시하																		
ERD, Auth Service	신의철																		
k8s 클러스터 구축	신의철																		
2단계: 이상탐지모듈, Proxy 컨테이너, Control Plane 개발																			
데이터 수집	이시하																		
중간보고서	공통																		
KD CNN + OCSVM	이시하																		
Proxy Container	정의진																		
Control Plane	신의철																		
3단계: 모니터링 대시보드 개발, 통합 테스트																			
백엔드 - 탐지 로그 수집	이시하																		
백엔드 - ERD, 웹소켓	정의진																		
프론트엔드 - 대시보드 개발	신의철																		
통합 테스트	공통																		
최종보고서	공통																		

7. 참고 문헌

¹ G. Yoon, J.-S. Kim, S. Kim, J. Jeong, M. Pratiwi, and Y.-H. Choi, "Lightweight Service Mesh for Intrusion Detection using KD-CNN in Cloud-Native Environments," in Proc. 2025 Cloud Comput. Secur. Workshop (CCSW 25), Taipei, Taiwan, 2025, pp. 1-13.